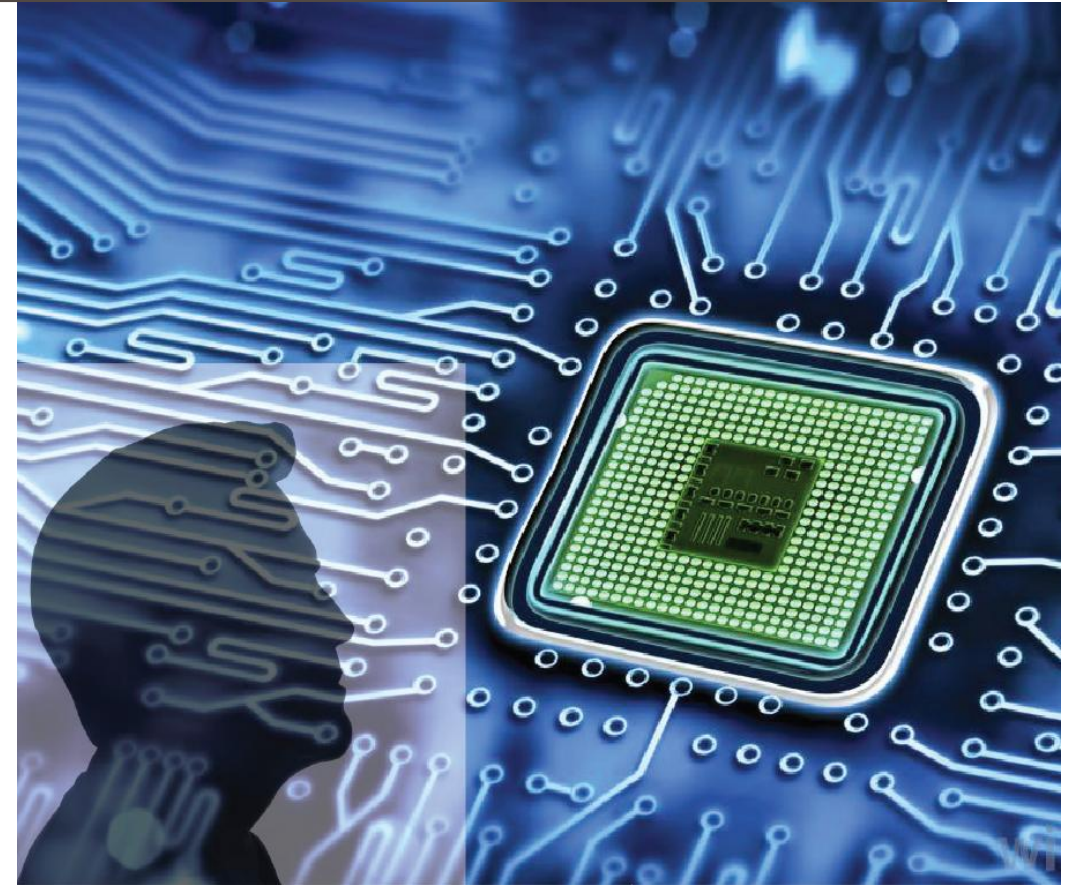


ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-7 Lecture-1 SystemVerilog

Sneh Saurabh
30th September, 2025



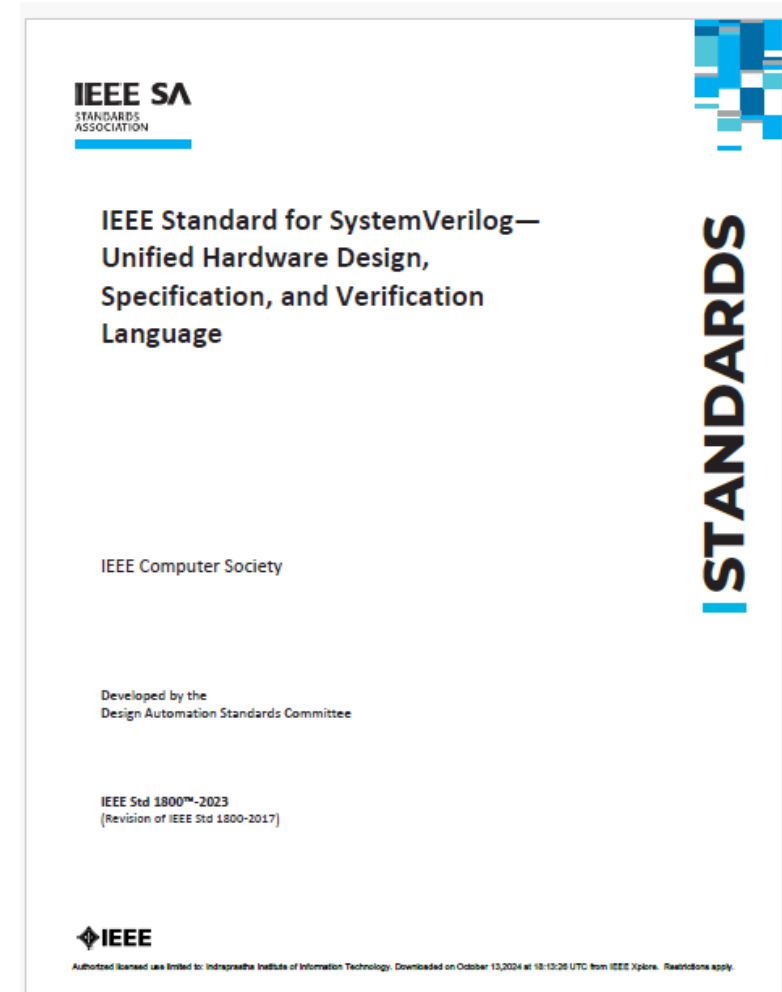
ADDV: Course Content

Week No.	Topic
1	Introduction; Digital Systems: Hardware and Software components; Software Development Process; Hardware Design and Verification Flow; Manufacturing and Test
2	Electronic System Level (ESL) Design: SystemC Models, Transaction-level Models (TLM), Virtual Platforms and Platform-based Design, Applications
3	Hardware-software co-design: models, design space exploration, partitioning, co-simulation
4	High-level Synthesis: Flow, Tasks, Control Data Flow Graph (CDFG), Optimizations
5	High-level Synthesis: Scheduling, Resource Allocation, Handling loops, arrays and functions, Pipelining
6	System-on-Chip (SoC) Design Methodology: IPs, IP Packaging, IP-XACT, IP Assembly, Generating RTL, Challenges
7	Introduction to SystemVerilog: Data Types, Improvements over Verilog for Synthesis and Simulation, Object-oriented Programming Constructs, Interfaces, Storage Elements, Inter-process Communications
8	Advanced Concepts on Simulation: compiled-simulators, random vector-based verification, constrained-random verification, coverage-driven verification
9	Advanced Concepts on Formal Verification: fundamentals of Property Checking, Assertion-based Verification, Bounded model Checking
10	Universal Verification Methodology (UVM); Scenario-based Software-driven Testing; Portable Test and Stimulus Standard (PSS); Hardware-assisted Verification
11	Clock Domain Crossing (CDC): Problem, Solutions, Verification; Reset Domain Crossing (RDC)
12	System-level Power Management and Optimizations: Dynamic Power Management, Policies, Implementation, Voltage Scaling, Software-level power management
13	Power Intent: Unified Power Format (UPF) and design flows

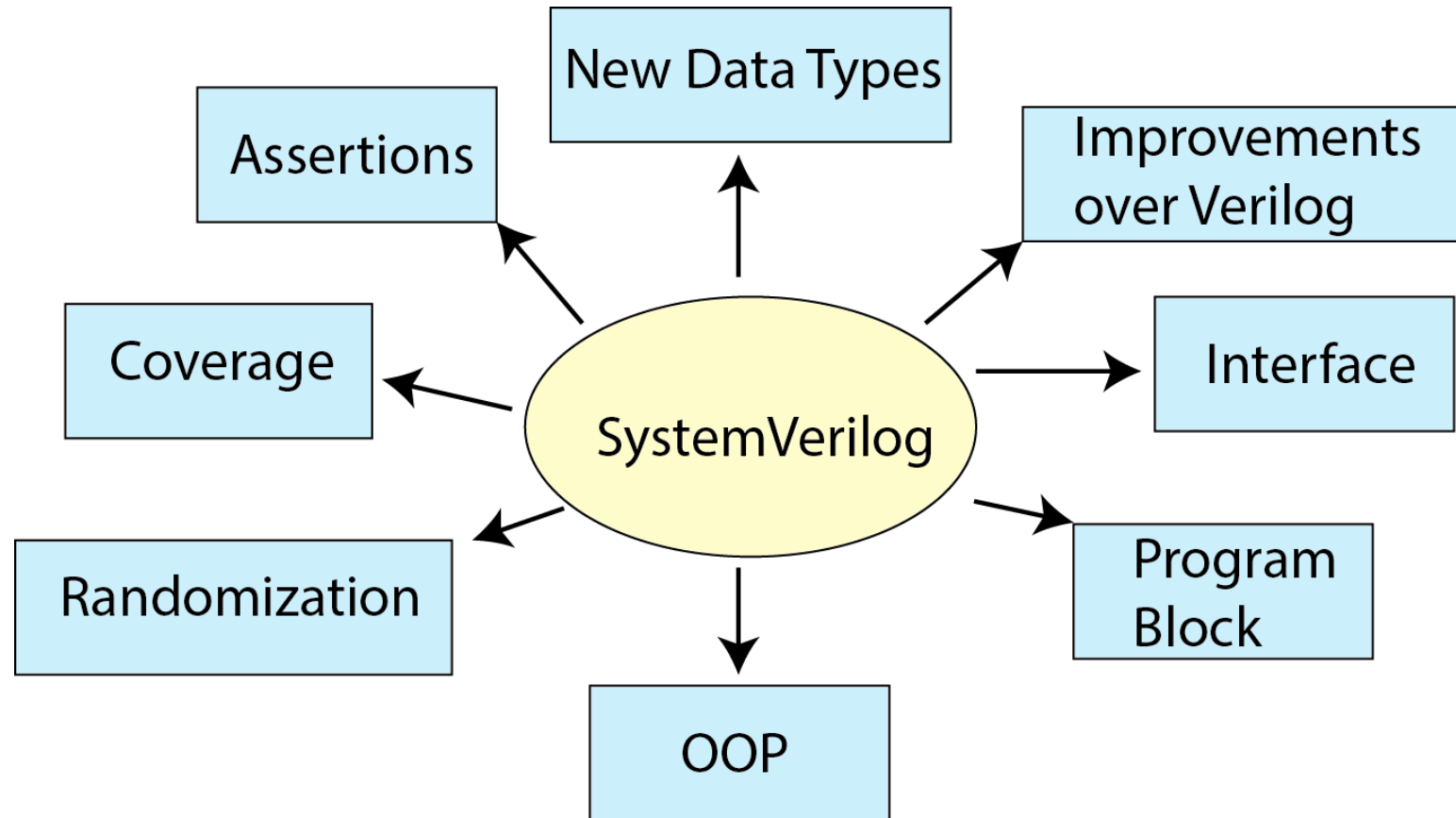
SystemVerilog

SystemVerilog: History

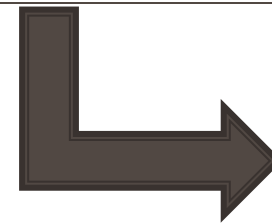
- Verilog was standardized by IEEE in 1995 and 2001
 - Features got added to support robust verification methodologies and a superset of Verilog known as SystemVerilog was developed
- The merged Verilog and SystemVerilog was standardized in 2009 (IEEE Standard 1800-2009)
 - SystemVerilog 2009 also ensures interoperability with other HDLs such as SystemC and VHDL
 - Latest version released in December 2023
- SystemVerilog is now widely used and supported by EDA tools to design, verify, synthesize, and test hardware.
 - In 2018, about 80% of integrated circuit (IC) design teams worldwide used Verilog or SystemVerilog



SystemVerilog: Components

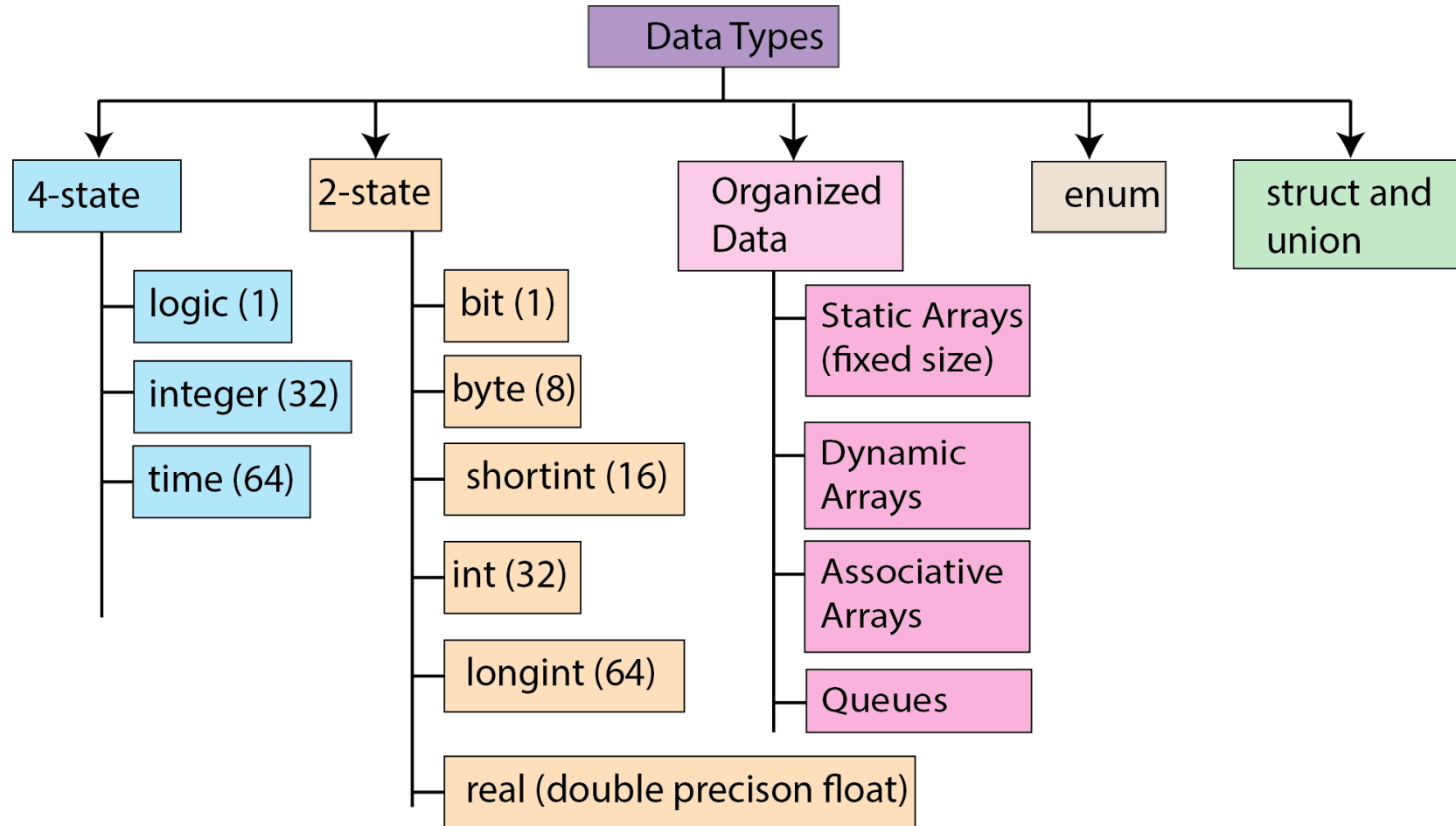


SystemVerilog

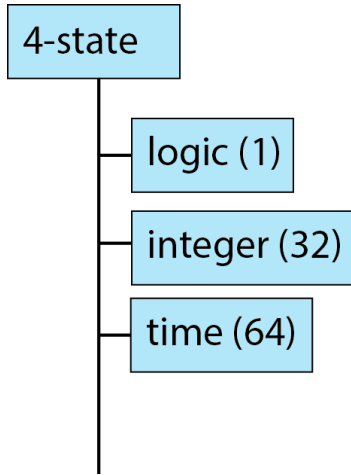


Data Types

Data Types: Overview



Data Types: 4-state



logic type:

- It is a combination of variable and net (**reg** and **wire** of Verilog)
 - Exception: **logic** variable cannot be driven by multiple structural drivers (need to use **wire** or **tri** in this case)

- \$isunknown()** : returns 1 if any bit of the expression is X or Z

```
if ($isunknown(iport)
    $display("@%0d: 4-state value detected on input port",
        $time, iport);
```

```
module logic_data_type(input logic rst_h);

    parameter CYCLE = 20;
    logic q, q_l, d, clk, rst_l;

    initial begin
        clk <= 0; // Procedural assignment
        forever #(CYCLE/2) clk = ~clk;
    end

    assign rst_l = ~rst_h; // Continuous assignment

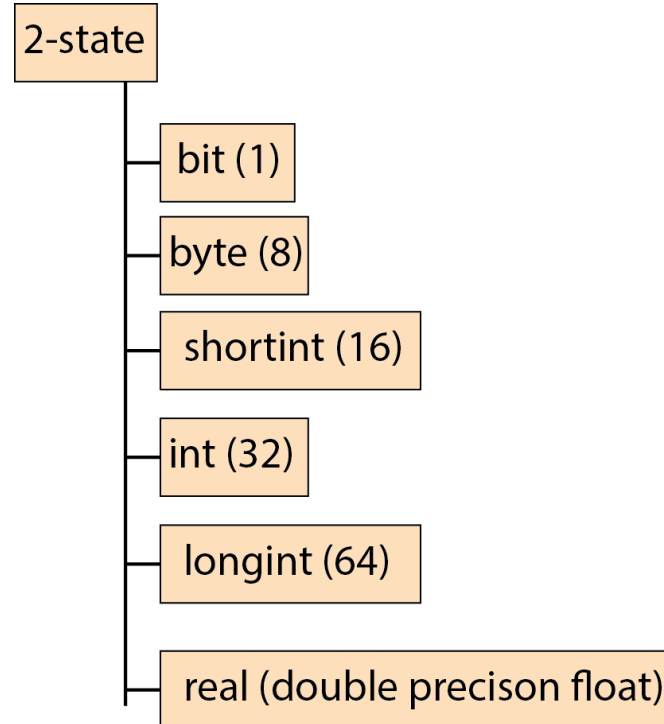
    not n1(q_l, q); // q_l is driven by gate

    my_dff d1(q, d, clk, rst_l); // d is driven by module

endmodule
```

```
integer i4; // 4-state, 32-bit signed integer
time t; // 4-state, 64-bit unsigned integer
```


Data Types: 2-state



- Improves performance of the simulator and reduces its memory usage
- Prefer these types over 4-state types (if possible)

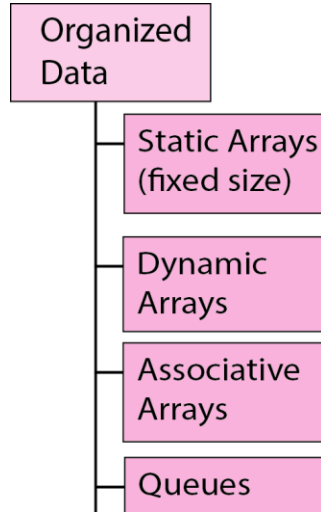
```
bit b; // 2-state, single-bit
bit [31:0] b32; // 2-state, 32-bit unsigned integer

byte b8; // 2-state, 8-bit signed integer

shortint s; // 2-state, 16-bit signed integer
int i; // 2-state, 32-bit signed integer
longint l; // 2-state, 64-bit signed integer

real r; // 2-state, double precision floating point
```

Data Types: Static Arrays



Enhancements in fixed-size arrays

- Can declare size rather than indices
- Can initialize using array literals
 - using an array literal that is an apostrophe and curly braces.
 - can set some or all elements at once.
 - can replicate values by putting a count before the curly braces
- **for** or **foreach** loop
 - **\$size** returns the size of the array
 - index variable is local to the loop

```
int lo_hi[0:15]; // 16 ints [0]..[15]
int c_style[16]; // 16 ints [0]..[15]
```

```
int array2 [0:7][0:3]; // Verbose declaration
int array3 [8][4]; // Compact declaration
array2[7][3] = 1; // Set last array element
```

```
int ascend[4] = '{0,1,2,3}; // Initialize 4 elements
int descend[5];
int md[2][3] = '{0,1,2}, {3,4,5}';
descend = '{4,3,2,1,0}; // Set 5 elements
descend[0:2] = '{5,6,7}; // Set first 3 elements
ascend = '{4{8}}; // Four values of 8
```

```
initial begin
    bit [31:0] src[5], dst[5];
    for (int i=0; i<$size(src); i++)
        src[i] = i;
    foreach (dst[j])
        dst[j] = src[j] * 2; // dst doubles src values
end
```

Data Types: Packed and Unpacked Arrays

- SystemVerilog stores each element on a longword (32-bit) boundary.
 - A byte, shortint, and int are stored in a single longword
 - A longint is stored in two longwords

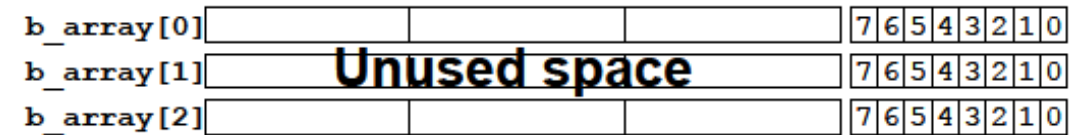
Packed arrays

- Packed bit and word dimensions are specified as part of the type, **before the variable name**
- Dimensions can be specified in the [msb:lsb] format, or [size]

When to use packed array?

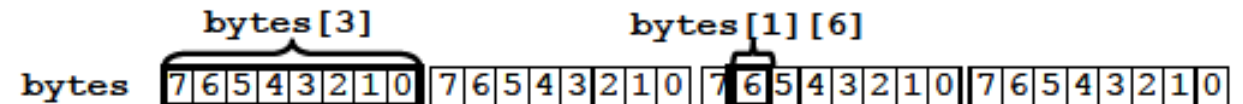
- If you need to convert to and from scalars
 - Example: need to reference a memory as a byte or as a longword
- Only fixed-size arrays can be packed

```
bit [7:0] b_unpacked[3]; // Unpacked
// Stored in three long words
```

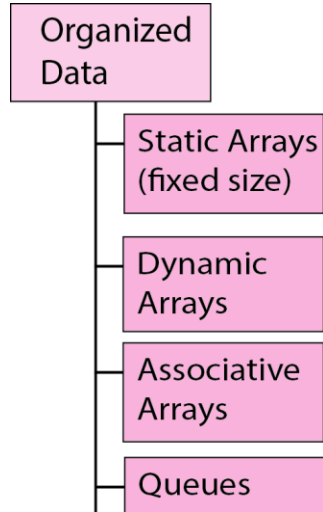


- Unpacked arrays have unused storage locations

```
bit [3:0] [7:0] bytes; // 4 bytes packed into 32-bits
bytes = 32'hdead_beef;
$displayh(bytes,, // Show all 32-bits
bytes[3], // most significant byte "de"
bytes[3][7]); // most significant bit "1"
```



Data Types: Dynamic Arrays



- Size decided during runtime
- Declared with empty word subscripts []
- Need to call the `new[]` operator to allocate space, passing in the number of entries in the square brackets
- Copying an array: pass the name of an array to the `new[]` operator

```
int dyn[], d2[]; // Empty dynamic arrays
initial begin
    dyn = new[5]; // Allocate 5 elements

    foreach (dyn[j])
        dyn[j] = j; // Initialize the elements

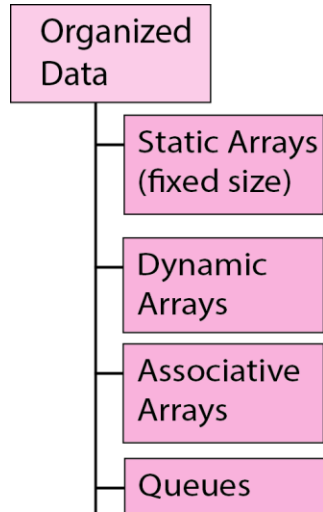
    d2 = dyn; // Copy a dynamic array

    d2[0] = 5; // Modify the copy
    $display(dyn[0],d2[0]); // See both values (0 & 5)

    dyn = new[20](dyn); // Expand and copy
    dyn = new[100]; // Allocate 100 new integers
    // Old values are lost

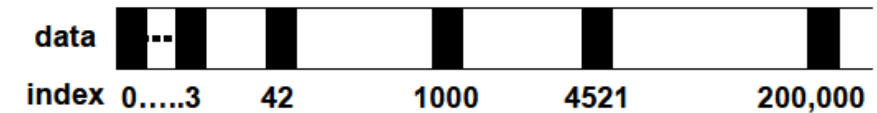
    dyn.delete; // Delete all elements
end
```

Data Types: Associative Arrays (1)



- Sometimes we need to model huge arrays
 - Example: modeling a processor that has a multi-gigabyte address range
 - Need to access only hundreds or thousands of location among GB-sized locations

- Associative arrays that store entries in a sparse matrix
 - Can address a very large address space, but allocates memory for an element when written to it



- Holds the values 0:3, 42, 1000, 4521, and 200,000.
- Does not need 200,000 entries as in a fixed or dynamic array

Data Types: Associative Arrays (2)

- Declared with wildcard syntax `[*]`
- Can use `foreach` to iterate
- Can use `first`, `next`, and `do-while` to iterate
 - `int first (ref key)` - assigns the key variable passed with first key of the associative array. If array is empty 0 is returned
 - `int next(ref key)` - assigns the key variable passed with the key which comes just after the specified key. If array is empty or if the specified key is last, 0 is returned.
- An associative array can be stored by the simulator as a tree
- Associative arrays can also be addressed with a string index
 - Reading from an element that has not been allocated yet, SystemVerilog returns a 0 for two-state types or X for 4-state types
 - Can use the function `exists` to check if an element exists

```
initial begin
    logic [63:0] assoc[*], idx = 1;

    // Initialize widely scattered values
    repeat (64) begin
        assoc[idx] = idx;
        idx = idx << 1;
    end

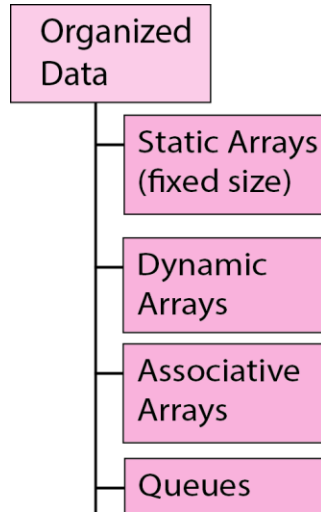
    // Step through all index values with foreach
    foreach (assoc[i])
        $display("assoc[%h] = %h", i, assoc[i]);

    // Step through all index values with functions
    if (assoc.first(idx))
        begin // Get first index
            do
                $display("assoc[%h]=%h", idx, assoc[idx]);
            while (assoc.next(idx)); // Get next index
        end

    // Find and delete the first element
    assoc.first(idx);
    assoc.delete(idx);

end
```

Data Types: Queues



Queues

- Can add and remove elements anywhere
 - Efficient to push and pop elements from the front and back [in $O(1)$]
 - Adding and deleting elements in the middle is slower
- Declared with wildcard `[$]`
- Like a dynamic array, queues can grow and shrink
 - Automatically allocates additional space as needed
 - Not needed to call the `new[]` operator for a queue

```
int j = 1, b[$] = {3,4}, q[$] = {0,2,5}; // {0,2,5} Initial queue

initial begin
    q.insert(1, j); // {0,1,2,5} Insert 1 before 2
    q.insert(3, b); // {0,1,2,3,4,5} Insert whole q.
    q.delete(1); // {0,2,3,4,5} Delete elem. #1

    // The rest of these are fast
    q.push_front(6); // {6,0,2,3,4,5} Insert at front
    j = q.pop_back; // {6,0,2,3,4} j = 5
    q.push_back(8); // {6,0,2,3,4,8} Insert at back
    j = q.pop_front; // {0,2,3,4,8} j = 6

    foreach (q[i])
        $display(q[i]);
end
```

Data Types: Enumerated Types

enum

- Enumerated types creates a strong variable type that is limited to a set of specified names
- create a named enumerated type to easily declare multiple variables (using **typedef**)

- Actual values default to integers starting at 0 and then increase.
 - Can choose enumerated values

- Can use the functions to step through types
 - **first**, **last**, **next**, **next(N)**, **prev**, **prev(N)**
- Functions **next** and **prev** wrap around when they reach the beginning or end of the enumeration

```
enum {RED, BLUE, GREEN} color;
```

```
// Create data type for values 0, 1, 2
typedef enum {INIT, DECODE, IDLE} fsmstate_e;
fsmstate_e pstate, nstate; // declare typed variables
```

```
initial begin
    case (pstate)
        IDLE: nstate = INIT; // data assignment
        INIT: nstate = DECODE;
        default: nstate = IDLE;
    endcase
    $display("Next state is %0s",
        nstate.name); // Use name function
end
```

```
typedef enum {INIT, DECODE=2, IDLE} fsmtype_e;

enum {RED, BLUE, GREEN} color;
color = color.first;
do
begin
    $display("Color = %0d/%0s", color, color.name);
    color = color.next;
end
while (color != color.first); // Done at wrap-around
```


Data Types: **struct** and **union**

struct

- Combine several variables into a structure
- **class** more versatile because it can have member functions also
 - consider using **class** rather than **struct**

```
typedef struct {bit [7:0] r, g, b;} pixel_s;  
pixel_s my_pixel;
```

union

- Are useful when we frequently need to read and write a register in several different formats
 - stores both the integer i and the real number f in the same location.
- Can save memory
 - Code becomes more complicated
 - consider whether using union is really needed

```
typedef union { int i; real f; } num_u;  
num_u un;  
un.f = 0.0; // set n in floating point format
```

Type Conversion

Static cast

- Converts between two types without checking
- Use destination type, apostrophe, and the expression

```
int i;  
real r;  
i = int '(10.0-0.4);  
r = 10.0-0.4;  
$display("i=%d r=%f", i, r);
```

Dynamic cast

- Converts between two types with checking (out of bounds)
- Use `$cast(destination, source)`
 - Returns 1 on success else 0

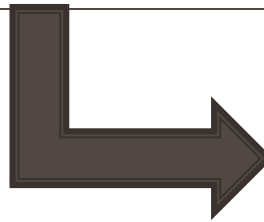
```
typedef enum {RED, BLUE, GREEN} COLOR_E;  
COLOR_E color, c2;  
integer c;  
initial begin  
    c = color; // Convert from enum to integer  
    c++; // Increment integer  
    if (!$cast(color, c)) // Cast integer back to enum  
        $display("Cast failed for c=%0d", c);  
    $display("Color is %0d / %0s", color, color.name);  
    c2 = COLOR_E'(c); // No type checking done  
end
```

Packages

- Allows sharing declarations among modules, packages, programs, and interfaces
- Use **package**, **endpackage** definition
- Use **import** for importing symbols from packages
- Use scope resolution operator **::** to avoid ambiguity

```
package ABC;  
    string message = "This is ABC";  
endpackage  
  
module top;  
    import ABC::*;  
  
    initial begin  
        $display("Message from package: %s", message);  
    end  
endmodule
```

SystemVerilog



Procedural Blocks and
Routines

SystemVerilog: Procedural Blocks

- SystemVerilog supports three additional types of always blocks.
 - `always_comb`: model combinational logic
 - `always_latch`: model latches
 - `always_ff`: model flip-flops
- Eliminate inadvertent design errors arising due to the generic always blocks
- EDA tools can infer sensitivity lists from the variables read in these blocks.

```
module proc_mod(input wire clk, rst, a, b, c, en,
               output reg op1, op2, op3);

    always_comb begin
        op1 = a + b;
    end

    always_latch begin
        if(en) begin
            op2 = c;
        end
    end

    always_ff @(posedge clk or negedge rst) begin
        if(!rst)
            op3 = 0;
        else begin
            op3 = a + b + c;
        end
    end

endmodule
```

References

- Spear, Chris. SystemVerilog for Verification: A Guide to Learning the Testbench Language Features. Vol. 161. Springer, 2008.